DMIOT模组MQTT通信协议

概述

本文档描述了设备与服务器之间的json传输方法，服务器可以通过订阅“up”主题来获取设备上报消息，通过发布“down”主题来下发消息。json协议里的method字段代表了该条消息类型。

RPC method	发起	描述
get_properties	server	读取设备属性，设备对应回复result附带属性值
set_properties	server	设置设备属性，设备对应回复result附带写入结果
properties_changed	device	设备变化上报属性
action	server	服务器通过方法控制设备
event_occured	device	事件发生设备主动上报
json_raw	server/device	服务器或设备MCU之间需要json直传可以调用该方法
information	device	设备信息改变或循环上报其信息
ota_process	device	设备处于OTA升级状态是会主动上报OTA进度
ota_start	server	服务器通知设备开始OTA升级
time	device	设备获取在线时间，刚联网或循环5分钟拉取
start_debug	server	进入调试模式
stop_debug	server	退出调试模式
get_spec	device	获取服务属性定义
upload_energy	device	设备主动上报能效数据

协议详述

一、读取设备属性get_properties

method方法为get_properties，参数信息为数组方式列出需要读取的属性其分组的服务号siid和属性号piid。设备在回复时会带上对应读取属性的错误码，如果错误码不为0则其值无效，请根据对应错误码确定报错原因。设备回复消息时没有method信息，所以请根据对应的消息号id来判断是对那条下发消息的回复。

示例：

服务器发送：

```
{
    "id" :2534,
    "method" : "get_properties",
    "params": [
    {
        "siid" : 1,
        "piid" : 2
    },
    {
        "siid" : 1,
        "piid" : 3
    }]
}
```

设备返回(成功)：

```
{
    "id" : 2534,
    "method" : "result",
    "params": [
    {
        "siid" : 1,
        "piid" : 2,
        "value": 10,
        "code" : 0
    },
    {
        "siid" : 1,
        "piid" : 3,
         "code" : -4003
    }]
}
```

设备返回(有失败)：

```
{
    "id" : 2534,
    "method" : "result",
    "params": [
    {
        "siid" : 1,
        "piid" : 2,
```

```
            "value": 10,
            "code" : 0
        },
        {
            "siid" : 1,
            "piid" : 3,
             "code" : -4001
        }]
}
```

二、设置设备属性set_properties

method方法为set_properties，参数信息为数组方式列出需要写入的属性的其分组服务号siid、属性号piid和对应的值。设备在执行完回复消息中带有错误码code，如果错误码不为0则写入存在异常，请根据对应错误码确定报错原因。设备回复消息时没有method信息，所以请根据对应的消息号id来判断是对那条下发消息的回复。

示例：

服务器发送：

```
{
        "id" :2534,
        "method" : "set_properties",
        params: [
        {
            "siid" : 1,
            "piid" : 1,
            "value":10
        },
        {
            "siid" : 1,
            "piid" : 88,
            "value":"hello"
        }]
}
```

设备返回成功：

```
{
        "id" : 2534,
        "method" : "result",
        "params": [
        {
            "siid" : 1,
            "piid" : 1,
            "code" : 0//成功
```

```
        },
        {
                "siid" : 1,
                "piid" :88,
                 "code" : -4003//属性不存在
        }]
}
```

三、服务器动作下发执行action

method方法为action，参数信息为数组方式列出需要写入的动作号的其分组服务号siid、动作号aiid和附带的属性号piid及对应的值。设备在执行完回复消息中带有错误码code，如果错误码不为0则执行异常，请根据对应错误码确定报错原因。设备回复消息时没有method信息，所以请根据对应的消息号id来判断是对哪条下发消息的回复。

示例：

服务器发送：

```
{
    "id": 52423,
    "method": "action",
    "siid": 1,
    "aiid": 1,
    "params": [
        {
            "piid": 2,
            "value": 10
        },
        {
            "piid": 3,
            "value": "low"
        }
    ]
}
```

设备返回：

```
{
        "id": 52423,
        "method" : "result",
        "result": {
         "code": 0,//动作执行成功
         "params": [{
           "piid": 2,
           "value": 10
```

```
        },
        {
          "piid": 3,
          "value": "low"
         }]
       }
}
```

四、设备主动上报属性properties_changed

method方法为properties_changed，参数信息为数组方式列出需要上报的属性其分组的服务号siid和属性号piid以及值。服务器无需对此消息进行回复。

示例：

服务器发送：

```
{
        "id" :2534,
        "method" : "properties_changed",
        params: [
        {
                "siid" : 1,
                "piid" : 2,
                "value": 17
        },
        {
                "siid" : 1,
                "piid" : 3,
                "value": "high"
        }]
}
```

五、设备上报事件event_occured

method方法为event_occured，参数信息为数组方式列出需要上报事件的服务号siid和事件号eiid，同时可以附带事件的一些属性参数值。服务器无需对此消息进行回复。

示例：

服务器发送：

```
{
        "id" :2534,
        "method" : "event_occured",
        "siid": 1,
        "eiid": 1,
        "params":
```

```
    [
        {
            "piid" : 2,
            "value": 17
        },
        {
            "piid" : 3,
            "value": "high"
        }
    ]
}
```


六、服务器/设备底层json消息直传json_raw

method方法为json_raw，附带的params子json就是需要相互直传的json包。

示例：

服务器/设备底层发送：


```
{
    "id" :2534,
    "method" : "json_raw",
    "params":
    {
        "msg_id":2093,
        "action":82,
        "resource":9003,
        "user_private":"hello world!"
    }
}
```


七、设备信息上报

method方法为information，附带的设备相关信息字段。WiFi如果有状态改变就会立即上报，同时也会循环1分钟上报一次。服务器无需回复。

示例：

设备发送：

JSON 复制代码

```json
{
  "id": 2534,
  "method": "information",
  "params": {
    "comm_version": "V2.0.1",
    "hardware_version": "esp32-wroon-32d",
    "library_version": "esp-idf-v4.0.1",
    "ssid": "dm_office",
    "local_ip": "192.168.0.102",
    "local_port": 8080,
    "mac": "46cd6455ee6f",
    "rssi": -63,
    "product_id": "5f44bb01fc46fc3b6c86de44",
    "product_model": "dm-mq100",
    "device_id": "645be89ca5343ceef",
    "sn": "46cd6455ee6f",
    "mcu_version": "V1.0.1",
    "slave_mcu": [
      {
        "type": 3,
        "version": "V1.0.0"
      },
      {
        "type": 4,
        "version": "V1.0.0"
      }
    ]
  }
}
```

八、设备OTA进度上报

method方法为ota_process，会通过state和rate两个参数上报进度。state:0-空闲，1-开始，2-正在下载，3-正在安装。rate取值0-100,单位%，代表OTA过程整体完成进度，0-50代表下载进度，51-100代表安装进度。

示例：

设备发送：

```
{
    "id" :2534,
    "method" : "ota_process",
    "params":
    {
        "state":3,
        "rate":87
    }
}
```

九、服务器开始OTA指令

method方法为ota_start，服务器下发指令，需要附带固件类型、版本号、固件长度、md5校验码。设备需要返回执行结果，code为0消息接收成功。

示例：

服务器发送：

```
{
    "id": 1126,
    "method": "ota_start",
    "params":
      {
          "firmware_type": 2,
         "firmware_version": "xf_v2.0.1",
          "file_length": 1233445,
         "md5": "dedea232d2c33d20"
    }
}
```

设备返回：

```
{
    "id": 1126,
    "method": "ota_start",
    "code":0
}
```

十、设备获取实时时间

method方法为time，设备定时通过此指令更新在线时间并同步到本地，服务器需要同步回复时间戳和时区，设备会自行转换为日期时间。

示例：

设备返回：

```
{
    "id": 1126,
    "method": "time",
}
```

服务器发送：

```
{
    "id": 1126,
    "method": "time",
    "params":
      {
```

```
            "timestamp": 1601281862,
            "timezone": 8.0
        }
}
```

十一、开始设备调试

method方法为 `start_debug` ，设备每秒上报一次调试数据到topic: `up/${productId}/${deviceId}/debug` ，调试数据内容为任意长度的字节数组，用于服务端调试设备运行情况。

示例：

服务器发送：

```
{
    "id": 1126,
    "method": "start_debug"
}
```

设备返回：

```
{
    "id": 1126,
    "method": "start_debug",
    "code":0
}
```

十二、停止设备调试

method方法为 `stop_debug` ，设备停止上报调试数据。

示例：

服务器发送：

```
{
    "id": 1126,
    "method": "stop_debug"
}
```

设备返回：

```
{
    "id": 1126,
    "method": "stop_debug",
    "code":0
}
```

十三、检查是否有固件待更新

method方法为firmware_check，设备上报固件更新检查，服务器下发线上最新固件版本和是否需要升级标志。

示例：

设备发送：

```
{
    "id":1126,
    "method":"firmware_check"
}
```

服务器返回：

```
{
    "id": 1126,
    "method": "firmware_check",
    "code":0,
    "firware":[
        {"type":1,"version":"V3.3.1","update":true},
        {"type":2,"version":"V1.1.1","update":false},
        {"type":3,"version":"V1.1.1","update":false},
        {"type":4,"version":"V11.1","update":false},
        {"type":5,"version":"V1.1.1","update":false},
        {"type":6,"version":"V1.1.1","update":false}
    ]
}
```

十四、请求OTA指令

method方法为ota_request，设备上报升级请求标志并带有需要升级的固件类型和目标固件版本，服务器回复成功，之后服务器通过ota_start指令让设备开始进行OTA升级。

示例：

设备发送：

```
{
    "id": 1126,
    "method": "ota_request",
    "params":
      {
          "firmware_type": 2,
          "firmware_version": "V3.3.1"
    }
}
```

服务器返回：

```
{
    "id": 1126,
    "method": "ota_request",
    "code":0
```

}

十五、获取SPEC

示例：

设备发送：

JSON 复制代码

```
{
  "id": "1",
  "method": "get_spec"
}
```

服务器返回：

JSON 复制代码

```json
{
  "id": 1,
  "method": "get_spec",
  "params": [
    {
      "serviceId": "63bbbfe173aa3b606324d7aa",
      "siId": 1,
      "name": "device_information",
      "remark": "设备信息",
      "properties": [
        {
          "propertyId": "63bbbfe173aa3b606324d7ab",
          "piId": 1,
          "name": "manufacturer",
          "remark": "厂家",
          "power": "read",
          "format": "string",
          "unit": "none",
          "type": "",
          "value": []
        },
        {
          "propertyId": "63bbbfe173aa3b606324d7ac",
          "piId": 2,
          "name": "model",
          "remark": "型号",
          "power": "read",
          "format": "string",
          "unit": "none",
          "type": "",
          "value": []
        }
      ]
    }
  ]
}
```

十六、设备主动上报能效数据

示例：

设备发送：

JSON 复制代码

```json
{
  "id": "1",
  "method": "upload_energy",
  "data": {
    "time": 1754037205,
    "interval": 3600,
    "3_1100": 123.4,
    "3_1101": 123.4,
    "...": 123.4,
  }
}
```

服务器返回:

JSON 复制代码

```json
{
  "id": "1",
  "method": "upload_energy_result",
  "code": 0
}
```

十七、服务器下发 让设备重哪天重新开始同步

示例:

设备发送:

JSON 复制代码

```json
{
  "id": "1",
  "method": "upload_energy_restart",
   "time":1754269200
}
```

服务器返回:

附录:

code状态码定义

| 0 | 成功 |

| 1 | 接收到请求，但操作还没有完成 |

| -4001 | 属性不可读 |

| -4002 | 属性不可写 |

| -4003 | 属性、方法、事件不存在 |

| -4004 | 其他内部错误 |

| -4005 | 属性 value错误 |

| -4006 | 方法 in 参数错误 |

| -4007 | did 错误